

Hamiltonian-Based Quantum Simulation of the 1 Dimensional Burgers' Equation

Internship Report

Submitted to

Naval Science & Technological Laboratory (NSTL), DRDO
Visakhapatnam



Submitted by

Gauri Sunil

Intern CFD division, HR Wing
NSTL, DRDO

July 2026

DECLARATION

I, Gauri Sunil, hereby declare that the work presented in this report entitled “Hamiltonian-Based Quantum Simulation of the 1D Burgers' Equation” was carried out by me during my internship at the CFD Division, HR Wing, Naval Science & Technological Laboratory (NSTL), DRDO, Visakhapatnam.

This report has been prepared under the guidance of Mr. V. F. Saji, Scientist–F and Head of the CFD Division, NSTL, DRDO. The technical approach, analysis, results, and conclusions presented in this report are derived from the work carried out by me in accordance with the objectives and responsibilities assigned during the internship.

All sources of information and reference materials used in the preparation of this report have been duly acknowledged.

Gauri Sunil

Intern CFD Division, HR Wing

NSTL, DRDO

Date: 31/07/26

Place: Visakhapatnam

ACKNOWLEDGEMENTS

I would like to take this opportunity to express my sincere gratitude to all those who supported and guided me throughout the course of my internship project.

I am deeply grateful to **Mr. V. F. Saji, Scientist–F, CFD Division, NSTL–DRDO**, for his invaluable guidance, encouragement, and insightful suggestions. His technical expertise and constant support were instrumental in shaping the direction and successful completion of this work.

I express my sincere appreciation to **all the scientists, engineers, and technical staff of the CFD Division, NSTL–DRDO**, for their guidance and timely assistance in addressing technical queries and clarifying doubts during the internship. Their willingness to share knowledge and provide support greatly enriched my learning experience.

I am also thankful to the **administrative and supporting staff of NSTL–DRDO** for providing a conducive and professional working environment, along with the necessary resources required to carry out this work effectively.

Finally, I extend my gratitude to **Naval Science & Technological Laboratory (NSTL), DRDO**, for providing me with the opportunity to undertake this internship and for offering a highly enriching research environment that contributed significantly to my technical and professional development.

TABLE OF CONTENTS

ABSTRACT.....	5
CHAPTER 1 INTRODUCTION	6
1.1 Background	6
1.2 Burgers' Equation.....	6
1.3 Problem Statement and Objectives	7
CHAPTER 2 LITERATURE SURVEY	8
2.1 Burgers' Equation.....	8
2.2 Classical Numerical Methods for PDEs	8
2.3 Cole–Hopf Transformation.....	9
2.4 Linear Combination of Unitaries (LCU).....	9
2.5 Research Gaps.....	9
CHAPTER 3 METHODOLOGY	10
3.1 Problem Formulation	10
3.2 Initial and Boundary Conditions.....	11
3.3 Classical Finite Difference Solver	11
3.4 Cole–Hopf Transformation.....	12
3.5 Hamiltonian Construction and Evolution	13
3.6 Linear Combination of Unitaries (LCU).....	13
3.7 Physics Informed Neural Network.....	14
3.8 Performance Evaluation.....	15
CHAPTER 4 RESULTS AND DISCUSSION.....	16
4.1 Stability Analysis.....	16
4.2 Cole–Hopf and Hamiltonian Validation	16
4.3 Comparative Performance Analysis	17
4.4 Limitations	18
4.5 Future Work.....	18
4.6 Final Remarks	19
APPENDICES	20
APPENDIX 1	20
APPENDIX 2.....	22
APPENDIX 3.....	23
APPENDIX 4.....	24
APPENDIX 5.....	25
APPENDIX 6.....	27
REFERENCES	28

ABSTRACT

The one-dimensional viscous Burgers' equation is a fundamental nonlinear partial differential equation widely used as a benchmark for studying computational fluid dynamics (CFD) and testing numerical solution techniques. This project investigates and compares three distinct computational approaches for solving the Burgers' equation: a classical finite difference method, a quantum-inspired method based on the Cole–Hopf transformation and Hamiltonian Simulation, and a Physics-Informed Neural Network (PINN). The simulation is performed on a one-dimensional domain, corresponding to a 5 qubit representation, with a Gaussian initial velocity profile and fixed boundary conditions.

The nonlinear Burgers' equation is first solved using a classical numerical solver to obtain a reference solution. The equation is then transformed into the linear heat equation using the Cole–Hopf transformation. The resulting diffusion operator is represented as a Hamiltonian and encoded using Qiskit's SparsePauliOp, allowing investigation of a segmented Linear Combination of Unitaries (LCU) approximation method. In parallel, a Physics-Informed Neural Network is trained by minimizing the governing PDE residual together with the initial and boundary condition losses, producing a mesh-free solution without explicit numerical discretization.

The accuracy of all methods is evaluated using the L2 error norm with respect to the classical reference solution. The quantum-inspired solution achieved an L2 error of approximately $5.030\text{e-}03$, while the PINN achieved an L2 error of approximately $4.990\text{e-}03$, demonstrating close agreement among the approaches. The project successfully demonstrates the application of Hamiltonian encoding, Pauli operator decomposition, and LCU techniques within the Qiskit framework. The work highlights the potential of quantum algorithms for solving PDEs while providing a comparative study of traditional numerical methods, quantum-inspired computation, and machine learning techniques for nonlinear fluid dynamics problems.

Keywords: *Partial differential equation, Burgers' equation, Quantum algorithm, Cole–Hopf transformation, Hamiltonian, Linear Combination of Unitaries, Physics Informed Neural Network*

CHAPTER 1

INTRODUCTION

1.1 Background

Computational Fluid Dynamics (CFD) is a branch of computational science that employs numerical methods to analyse and simulate fluid flow by solving the governing equations of fluid motion. The equations governing fluid flow, particularly the Navier–Stokes equations, are highly nonlinear and often difficult to solve analytically. As a result, numerical methods have become the primary approach for studying fluid dynamics problems.

The one-dimensional viscous Burgers' equation is one of the most widely used benchmark equations in CFD. Recent developments have introduced alternative approaches for solving these PDEs beyond traditional numerical techniques. Machine learning methods such as Physics-Informed Neural Networks (PINNs) enable the direct approximation of PDE solutions by embedding the governing physical laws into the neural network training process. Similarly, advances in quantum computing have motivated the development of quantum algorithms capable of solving large-scale linear systems and simulating physical processes more efficiently than classical algorithms under suitable conditions.

1.2 Burgers' Equation

The one-dimensional viscous Burgers' equation,

$$\frac{\partial u}{\partial t} + u \frac{\partial u}{\partial x} = \nu \frac{\partial^2 u}{\partial x^2},$$

is widely regarded as the simplest nonlinear PDE that retains the essential structure of the Navier–Stokes momentum equation while dropping the pressure term and the incompressibility constraint. It combines a nonlinear convective term, uu_x , with a linear diffusive term, νu_{xx} , and can produce steep, near-shock gradients when the viscosity ν is small, while remaining smooth and well-behaved for moderate viscosity. This makes it an ideal test bed for comparing numerical, quantum-inspired, and machine-learning solution strategies: it is nonlinear enough to be a meaningful stress test, yet, crucially, it possesses an exact linearizing transformation – the Cole-Hopf transformation – that makes it uniquely suited to quantum Hamiltonian simulation techniques, which are fundamentally linear in nature.

1.3 Problem Statement and Objectives

The numerical solution of nonlinear partial differential equations remains computationally demanding. Classical numerical methods require spatial and temporal discretization, leading to increasing computational cost as the problem size grows.

The effectiveness of currently emerging alternatives must first be evaluated on benchmark problems before they can be applied to more complex fluid dynamics simulations. The Burgers' equation provides an ideal test case for comparing these different computational approaches under identical physical conditions.

The primary objectives of this project are:

- To implement a classical finite difference solver as a reference solution.
- To transform the nonlinear Burgers' equation into the linear heat equation using the Cole–Hopf transformation.
- To construct the corresponding Hamiltonian representation of the diffusion operator and implement exact Hamiltonian evolution using matrix exponentiation as the reference Hamiltonian solution.
- To implement a segmented Linear Combination of Unitaries (LCU) algorithm.
- To implement a Physics-Informed Neural Network (PINN) capable of solving the Burgers' equation directly from its governing physics.
- To compare the performance and accuracy of the classical numerical method, quantum-inspired Hamiltonian methods, and PINN using the L2 error norm.

CHAPTER 2

LITERATURE SURVEY

2.1 Burgers' Equation

The one-dimensional viscous Burgers' equation contains two competing physical mechanisms. Convection (advection), which is represented by the nonlinear term $u \partial u / \partial x$, is responsible for transporting momentum, and diffusion, which is represented by $\nu \partial^2 u / \partial x^2$, smooths velocity gradients due to viscosity.

2.2 Classical Numerical Methods for PDEs

Analytical solutions for nonlinear PDEs are generally available only for special cases. Consequently, numerical methods are commonly employed to approximate their solutions.

Finite Difference Methods (FDM) replace spatial and temporal derivatives by algebraic approximations on a discrete computational grid. The computational domain is divided into a finite number of grid points, and the governing equation is solved iteratively over time.

Several numerical schemes have been developed for solving Burgers' equation, including:

- Forward Time Central Space (FTCS)
- Upwind schemes
- Crank–Nicolson method
- Runge–Kutta based finite difference methods

Explicit schemes are simple to implement but require stability conditions such as the Courant–Friedrichs–Lewy (CFL) criterion. Implicit methods offer improved stability but require solving systems of equations at every time step.

2.3 Cole–Hopf Transformation

The nonlinear convection term is the primary difficulty in solving Burgers' equation. In 1950, Cole and Hopf independently introduced a mathematical transformation that converts the nonlinear Burgers' equation into the linear heat equation (the exact form of the transformation and its substitution into Burgers' equation is given in Section 3.4).

Since the heat equation is linear, it can be solved using matrix methods, spectral techniques, or Hamiltonian simulation methods. After obtaining the solution of the heat equation, the inverse Cole–Hopf transformation is applied to recover the Burgers' solution. The Cole–

Hopf transformation provides the mathematical foundation for the quantum-inspired component of this project.

2.4 Linear Combination of Unitaries (LCU)

Hamiltonian simulation is one of the most important applications of quantum computing. The Linear Combination of Unitaries (LCU) algorithm is a widely studied method for approximating matrix exponentials by expanding them as a truncated Taylor series, in which each power of the Hamiltonian is decomposed into a weighted sum of unitary Pauli operators (the expansion used in this project is given in Section 3.6).

Rather than implementing the exponential directly, the LCU algorithm prepares a quantum state that probabilistically applies these unitary operators according to their corresponding coefficients, using ancillary qubits to encode the coefficients while controlled unitary operations implement the Hamiltonian evolution. In practice, the Taylor series is truncated to a finite order, introducing approximation error; to improve numerical stability, the total time evolution is often divided into multiple smaller segments, with the truncated series applied independently to each segment. Although executed through classical simulation in this project, the LCU algorithm closely follows the mathematical structure of its quantum counterpart.

2.5 Research Gaps

The literature survey indicates that although significant progress has been made in solving Burgers' equation using classical numerical methods, quantum algorithms, and machine learning techniques, there is limited research that combines these approaches within a single comparative framework. Most existing studies focus either on improving classical numerical solvers or developing theoretical quantum algorithms without providing practical implementations. Similarly, while the Cole–Hopf transformation has been extensively studied as a mathematical technique for linearizing Burgers' equation, relatively few works integrate this transformation with Hamiltonian construction and quantum computing frameworks such as Qiskit.

CHAPTER 3

METHODOLOGY

This chapter describes the three computational pipelines implemented for this project: the classical finite difference reference solver, the Cole–Hopf/Hamiltonian/LCU quantum-inspired route, and the Physics-Informed Neural Network. Figure 1 summarizes how the three approaches relate to one another and converge on a common L2 error comparison.

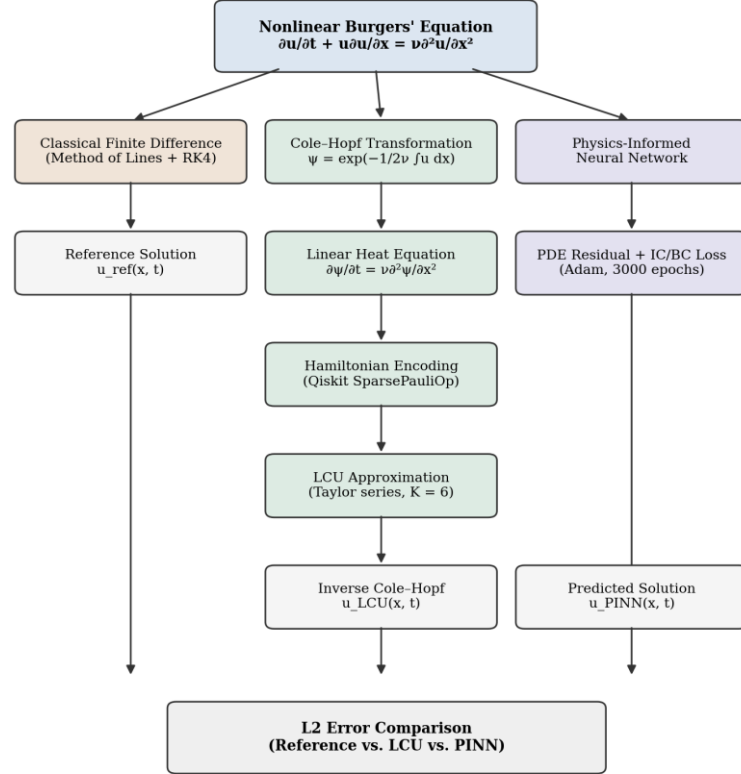


Figure 1: Overview of the classical, quantum-inspired, and PINN solution pipelines

3.1 Problem Formulation

The governing equation considered in this project is the one-dimensional viscous Burgers' equation

$$\frac{\partial u}{\partial t} + u \frac{\partial u}{\partial x} = \nu \frac{\partial^2 u}{\partial x^2},$$

where

- $u(x, t)$ is the velocity field,
- x denotes the spatial coordinate,
- t represents time,
- ν is the kinematic viscosity.

The computational domain is defined as

$$0 \leq x \leq 1.$$

The numerical parameters used throughout the project are summarized below.

Parameter	Value
Domain Length	1.0
Number of Grid Points	32
Number of Qubits	5
Spatial Step (Δx)	0.03125
Final Time	0.3 s
Time Step	0.001 s
Number of Time Steps	300
Viscosity (ν)	0.05

3.2 Initial and Boundary Conditions

The initial velocity profile is chosen as a Gaussian pulse centered within the computational domain.

$$u(x, 0) = e^{-50(x-0.5)^2}$$

This smooth initial condition allows the effects of nonlinear convection and viscous diffusion to be clearly observed during the simulation.

Dirichlet boundary conditions are imposed throughout the simulation:

$$u(0, t) = 0, \quad u(L, t) = 0$$

The same physical conditions are maintained for all computational methods to ensure a fair comparison.

3.3 Classical Finite Difference Solver

The classical solution of the one-dimensional viscous Burgers' equation was implemented using the Method of Lines (MOL), in which the spatial derivatives are first discretized using finite difference approximations, converting the governing partial differential equation into a system of ordinary differential equations (ODEs). These ODEs are then integrated in time using the classical fourth-order Runge–Kutta (RK4) method. This higher-order approach was adopted to provide a more accurate reference solution for comparison with the quantum-inspired and Physics-Informed Neural Network (PINN) methods.

Before the simulation, numerical stability was verified using the Courant–Friedrichs–Lewy (CFL) conditions for both the advection and diffusion terms. The advection and diffusion CFL numbers were computed as

$$\text{CFL}_{\text{adv}} = \max(|u|)\Delta t / \Delta x, \quad \text{CFL}_{\text{diff}} = \nu \Delta t / \Delta x^2$$

Python assertion statements ensure these values remain within stable limits before execution, preventing unstable numerical behaviour.

The nonlinear convection and diffusion terms were discretized using fourth-order central finite difference schemes for both the first and second spatial derivatives. Compared to the earlier first-order upwind implementation using FTCS, this significantly reduces truncation error and minimizes artificial numerical diffusion. Since fourth-order stencils cannot be applied directly near the boundaries, one-sided second-order finite difference approximations were used at the boundary and near-boundary grid points, eliminating the need for ghost cells.

The spatial discretization converts Burgers' equation into the system

$$du/dt = -u \partial u / \partial x + \nu \partial^2 u / \partial x^2$$

which is evaluated through the `rhs()` function. Time integration is then performed using the RK4 algorithm, where four intermediate slopes (k_1 , k_2 , k_3 , and k_4) are computed at each time step and combined through weighted averaging. This increases the temporal accuracy from first order to fourth order, while improving stability over the Forward Euler method.

The numerical solution at every time step was stored in the history array (`u_hist`), producing a complete space-time solution used for visualization and as the reference solution for comparison. The improved RK4 solution was also compared with the earlier FTCS implementation using the L2 error norm, checking the increase in numerical accuracy achieved.

3.4 Cole–Hopf Transformation

The nonlinear Burgers' equation is converted into a linear heat equation using the Cole–Hopf transformation.

The forward transformation is

$$\psi = \exp\left(-\frac{1}{2\nu} \int_0^x u(s) ds\right)$$

where the spatial integral is evaluated numerically using the trapezoidal rule.

After substitution, Burgers' equation becomes

$$\partial \psi / \partial t = \nu \partial^2 \psi / \partial x^2$$

The inverse transformation used after solving the heat equation is

$$u = -2v \partial/\partial x (\ln \psi)$$

where the spatial derivative is computed using second-order finite differences.

This transformation allows the nonlinear problem to be solved through linear Hamiltonian evolution.

3.5 Hamiltonian Construction and Evolution

The heat equation is discretized using finite differences, producing a tridiagonal diffusion matrix.

Each interior point is represented as

$$\partial^2 \psi / \partial x^2 \approx (\psi_{i+1} - 2\psi_i + \psi_{i-1}) / \Delta x^2$$

The resulting matrix represents the diffusion operator.

To enable quantum-style computation, this matrix is converted into a Pauli operator representation using Qiskit's SparsePauliOp class.

The solution of the heat equation is then expressed as

$$\psi(T) = e^{HT} \psi_0$$

where

- H is the Hamiltonian,
- ψ_0 is the transformed initial state.

The initial state is normalized before evolution to satisfy quantum state requirements.

The matrix exponential

$$e^{HT}$$

is evaluated exactly using SciPy's matrix exponential function for now.

After time evolution, the original norm is restored and the inverse Cole–Hopf transformation is applied to recover the Burgers' solution.

This method provides the reference Hamiltonian solution used for validating the LCU approximation.

3.6 Linear Combination of Unitaries (LCU)

Instead of computing the matrix exponential exactly, the LCU algorithm approximates Hamiltonian evolution using a truncated Taylor series.

The exponential operator is expanded as

$$e^{Ht} = \sum_{k=0}^K (Ht)^k / k!$$

where

- Taylor order = 6
- Number of time segments = 30

Each Hamiltonian power is decomposed into weighted Pauli operators.

The algorithm sequentially applies these approximations over multiple small time intervals to improve numerical stability.

The output state is then rescaled before applying the inverse Cole–Hopf transformation.

This implementation follows the mathematical framework used by quantum LCU algorithms and demonstrates how Hamiltonian evolution can be approximated using quantum computing principles.

3.7 Physics Informed Neural Network

A Physics-Informed Neural Network is implemented as an entirely independent solution method.

The network accepts two inputs,

$$(x, t)$$

and predicts the corresponding velocity

$$u(x, t).$$

The architecture consists of

- 4 hidden layers
- 64 neurons per layer
- Tanh activation function

Automatic differentiation computes u_t , u_x , u_{xx} directly from the neural network.

The loss function combines PDE residual loss, initial condition loss and boundary condition loss. The network is optimized using the Adam optimizer.

Unlike the finite difference solver, the PINN does not require spatial discretization or explicit time stepping.

3.8 Performance Evaluation

The accuracy of every method is evaluated using the L2 error norm,

$$L_2 = \sqrt{\frac{1}{N} \sum_{i=1}^N (u_i^{ref} - u_i)^2}$$

where

- u_{ref} is the classical finite difference solution,
- u is the solution obtained from the corresponding method.

The methods compared are

- Classical Finite Difference Solver
- Quantum LCU Approximation
- Physics-Informed Neural Network

The final velocity profiles are also compared graphically to evaluate the agreement between different computational approaches.

CHAPTER 4

RESULTS AND CONCLUSIONS

4.1 Stability Analysis

Before the numerical simulation, the stability of the classical solver was verified using the Courant–Friedrichs–Lewy (CFL) conditions. The obtained values were

- Advection CFL: 0.0320
- Diffusion CFL: 0.0512

Both values are significantly below the stability limits adopted in the implementation, confirming that the selected spatial and temporal discretization parameters provide a stable numerical solution. Consequently, the observed differences between the solution methods are attributed primarily to the numerical formulations rather than instability in the simulation.

The resulting velocity profile exhibits the expected physical behaviour of the Burgers' equation, where nonlinear convection transports the Gaussian pulse while viscosity smooths the solution over time.

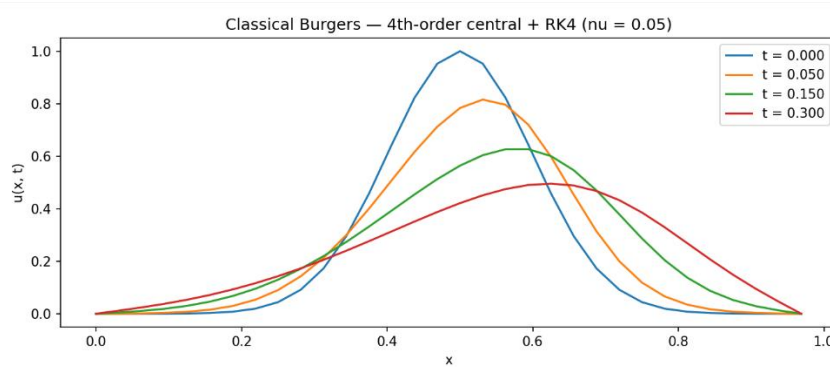


Figure 2: classically solved burgers' equation

4.2 Cole–Hopf and Hamiltonian Validation

The correctness of the Cole–Hopf implementation was verified by comparing the transformed heat equation solution with the classical Burgers' equation solution after applying the inverse transformation. The obtained L2 error was 7.10×10^{-3} .

This low error demonstrates that the nonlinear Burgers' equation was successfully transformed into the linear heat equation and reconstructed with high accuracy.

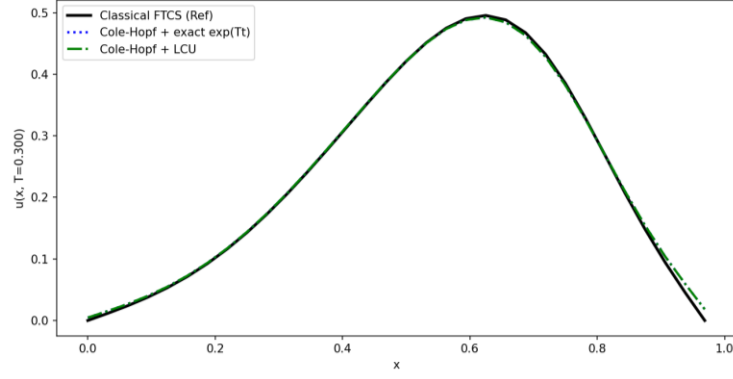


Figure 1: Validating proper implementation of LCU by matching exact $\exp()$ with LCU implimentation

The reconstruction error of 4.26×10^{-14} is effectively at machine precision, indicating that the Pauli decomposition accurately reproduces the original finite difference operator.

4.3 Comparative Performance Analysis

The final solutions obtained using the quantum-inspired LCU solver and the Physics-Informed Neural Network were compared with the classical finite difference reference using the L2 error norm.

Method	L2 Error
Classical MOL + RK4 (Reference)	0.000
Cole–Hopf + LCU (K = 6)	5.030×10^{-3}
Physics-Informed Neural Network	4.990×10^{-3}

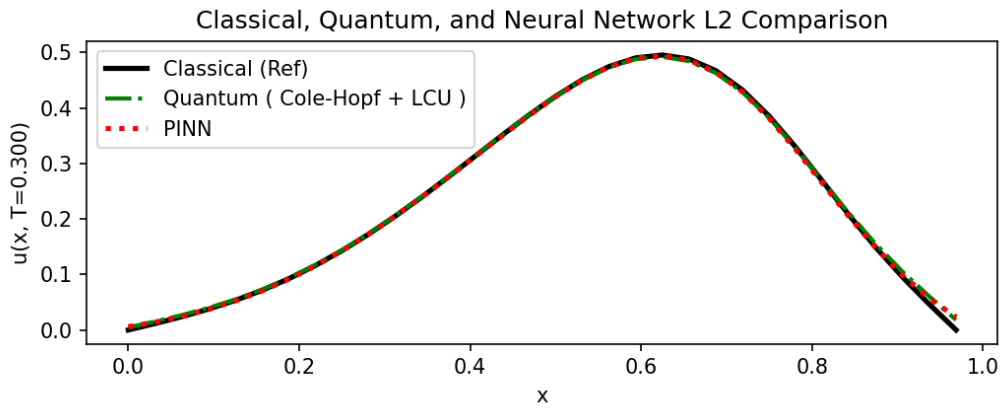


Figure 1: Comparison plot between classical, quantum and PINN solutions

The solution profiles obtained from the three approaches show excellent agreement over the entire computational domain. The quantum-inspired LCU solution closely follows the classical reference because the Cole–Hopf transformation converts the nonlinear Burgers' equation into a linear heat equation, enabling accurate Hamiltonian evolution before

reconstructing the original solution. Similarly, the PINN accurately captures both the nonlinear convection and diffusion behaviour by minimizing the governing PDE residual during training.

Minor differences are observed only near regions of high solution gradients, where discretization errors in the finite difference method and optimization errors in the neural network are most significant. Nevertheless, these differences remain very small, as reflected by the low L2 errors.

4.4 Limitations

The present work is subject to some limitations:

- The study is limited to the one-dimensional viscous Burgers' equation and does not consider higher-dimensional or more complex fluid dynamics problems.
- The spatial discretization consists of 32 grid points (5-qubit representation), limiting the achievable spatial resolution.
- The Hamiltonian evolution is performed using classical simulation rather than execution on physical quantum hardware.
- The PINN implementation employs a relatively simple fully connected neural network architecture trained for a fixed number of epochs, leaving scope for further optimization.

4.5 Future Work

The present project provides a foundation for several future directions.

- Future work may extend the methodology to two-dimensional and three-dimensional Burgers' equations and eventually to the incompressible Navier–Stokes equations.
- The current Hamiltonian evolution and LCU methods are executed using classical simulation. Future work may involve implementing the complete quantum circuits and executing them on a quantum computer to evaluate the effects of gate noise, decoherence, and limited qubit connectivity.
- The PINN implementation can be enhanced through deeper neural network architectures, hybrid optimization methods, and improved loss weighting techniques to accelerate convergence and improve prediction accuracy.

- Future studies may include detailed benchmarking of computational time, memory consumption, and algorithmic complexity across classical, quantum-inspired, and machine learning methods.

4.6 Final Remarks

Overall, the close agreement between the classical, quantum-inspired, and neural-network approaches confirms the correctness of the proposed methodology and demonstrates that Hamiltonian-based quantum methods and Physics-Informed Neural Networks are viable alternative strategies for solving nonlinear PDEs, achieving accuracy comparable to a high-order classical solver. Hamiltonian-based methods offer a mathematically elegant route into quantum computation; and PINNs provide a mesh-free alternative that solves the governing equations directly through optimization. The study validates the proposed implementation and highlights the potential of quantum-inspired and machine learning techniques as promising alternatives for solving nonlinear partial differential equations.

APPENDICES

APPENDIX 1

compare.py

<pre> from params import * import numpy as np u0 = u_init(x) CFL_adv = np.max(np.abs(u0)) * dt / dx CFL_diff = NU * dt / dx**2 print(f'CFL advection : {CFL_adv:.4f}') print(f'CFL diffusion : {CFL_diff:.4f}') assert CFL_adv < 1.0, 'Increase N_STEPS to reduce dt' #if it evaluates to False, Python immediately halts the program and raises an AssertionError assert CFL_diff < 0.9, 'Increase N_STEPS to reduce dt' def dudx_4th(u): """4th-order central first derivative, one- sided 2nd/3rd-order stencils at the two nearest-boundary points so nothing needs ghost cells.""" d = np.zeros_like(u) """du/dt = -u u_x + nu u_xx (non- conservative form, matches classical.py)""" return -u * dudx_4th(u) + nu * d2udx2_4th(u) </pre>	<pre> d[2:-2] = (-u[4:] + 8*u[3:-1] - 8*u[1:-3] + u[: -4]) / (12*dx) d[1] = (u[2] - u[0]) / (2*dx) # 2nd-order d[-2] = (u[-1] - u[-3]) / (2*dx) d[0] = (-3*u[0] + 4*u[1] - u[2]) / (2*dx) # one-sided d[-1] = (3*u[-1] - 4*u[-2] + u[-3]) / (2*dx) return d def d2udx2_4th(u): """4th-order central second derivative, 2nd- order one-sided near boundary.""" d = np.zeros_like(u) d[2:-2] = (-u[4:] + 16*u[3:-1] - 30*u[2:-2] + 16*u[1:-3] - u[: -4]) / (12*dx**2) d[1] = (u[2] - 2*u[1] + u[0]) / dx**2 # 2nd-order d[-2] = (u[-1] - 2*u[-2] + u[-3]) / dx**2 d[0] = (u[2] - 2*u[1] + u[0]) / dx**2 # reuse neighbour stencil d[-1] = (u[-3] - 2*u[-2] + u[-1]) / dx**2 return d </pre>
--	---

<pre> def apply_bc(u): u[0] = 0.0 u[-1] = 0.0 return u def solve_burgers_v2(u0, nu=NU): u = u0.copy() history = [u.copy()] for _ in range(N_STEPS): k1 = rhs(u, nu) k2 = rhs(apply_bc(u + 0.5*dt*k1), nu) k3 = rhs(apply_bc(u + 0.5*dt*k2), nu) k4 = rhs(apply_bc(u + dt*k3), nu) u = u + (dt/6.0) * (k1 + 2*k2 + 2*k3 + k4) u = apply_bc(u) history.append(u.copy()) return np.array(history) </pre>	<pre> u_hist = solve_burgers_v2(u_init(x)) if __name__ == "__main__": import matplotlib.pyplot as plt plt.figure(figsize=(9, 4)) for t_idx in [0, 50, 150, 300]: if t_idx < len(u_hist): plt.plot(x, u_hist[t_idx], label=f't = {t_idx*dt:.3f}') plt.xlabel('x'); plt.ylabel('u(x, t)') plt.title(f'Classical Burgers — 4th-order central + RK4 (nu = {NU})') plt.legend(); plt.tight_layout() plt.show() # compare against old FTCS from classical import u_hist as u_hist_ftcs l2 = np.sqrt(np.mean((u_hist[-1] - u_hist_ftcs[-1])**2)) print(f'L2 difference vs old FTCS solver: {l2:.2e}') </pre>
--	--

APPENDIX 2

colehopf.py

```
from scipy.integrate import
cumulative_trapezoid

from params import *

from classical import *

def cole_hopf_forward(u0, nu=NU):

    integral = cumulative_trapezoid(u0, dx=dx,
initial=0.0)

    psi0 = np.exp(-integral / (2.0 * nu))

    return psi0

def solve_heat(psi0, nu=NU):

    psi = psi0.copy()

    history = [psi.copy()]

    coeff = nu * dt / dx**2

    for _ in range(N_STEPS):

        psi_new = psi.copy()

        # Update interior points only

        psi_new[1:-1] = psi[1:-1] + coeff *
(psi[2:] - 2*psi[1:-1] + psi[:-2])

        # Dirichlet BC: hold boundary values
fixed at their t=0 values

        psi_new[0] = psi_new[1]

        psi_new[-1] = psi_new[-2] # fixed at
initial value

        psi = psi_new

        history.append(psi.copy())

    return np.array(history)

def cole_hopf_inverse(psi, nu=NU):

    #Inverse Cole-Hopf

    logpsi = np.log(np.maximum(psi,1e-14))

    dlogpsi = np.gradient(logpsi,
dx,edge_order=2)

    return -2.0*nu*dlogpsi

psi0 = cole_hopf_forward(u_init(x))

# Roundtrip check at t=0

u0_rt = cole_hopf_inverse(psi0)

l2_rt = np.sqrt(np.mean((u_init(x) - u0_rt)**2))

#print(f"Roundtrip L2 at t=0: {l2_rt:.2e} ")

psi_hist = solve_heat(psi0)

u_ch_final = cole_hopf_inverse(psi_hist[-1])

l2_full = np.sqrt(np.mean((u_hist[-1] -
u_ch_final)**2))

l2_int = np.sqrt(np.mean((u_hist[-1][2:-2] -
u_ch_final[2:-2])**2))

print(f"L2 error : {l2_full:.2e}")
```

APPENDIX 3

hamiltonian.py

```

from params import *

from qiskit.quantum_info import
SparsePauliOp

import numpy as np

def build_diffusion_hamiltonian(n_qubits,
dx, nu):
    """
    Build the FD Laplacian as
    SparsePauliOp.

    T[i,i] = -2 * nu/dx^2

    T[i,i+1] = T[i+1,i] = nu/dx^2

    Neumann BC: corner elements are -
    nu/dx^2 (one neighbour only).
    """
    n = 2**n_qubits

    c = nu / dx**2      # off-diagonal
    coefficient

    T = np.zeros((n, n))

    for i in range(n):

        T[i, i] = -2.0 * c

        if i > 0:    T[i, i-1] = c

```

```

        if i < n-1: T[i, i+1] = c

        # Neumann BC: boundary nodes have
        only one interior neighbour

        T[0, 0] = -c

        T[-1, -1] = -c

        # Decompose into Pauli basis

        H = SparsePauliOp.from_operator(T)

        H = H.simplify(atol=1e-10) # drop
        negligible terms

        return H, T

H, T_matrix =
build_diffusion_hamiltonian(N_QUBITS,
dx, NU)

H_recovered = H.to_matrix().real

max_err = np.max(np.abs(H_recovered -
T_matrix))

print(f'Hamiltonian reconstruction error:
{max_err:.2e}') # expect < 1e-12

print(f'Number of Pauli terms: {len(H)}')

```

APPENDIX 4

lcu.py

<pre> from params import * from hamiltonian import H, T_matrix from coleshopf import cole_hopf_forward, cole_hopf_inverse from classical import u_hist import numpy as np from qiskit.quantum_info import SparsePauliOp def taylor_lcu_terms(H_pauli: SparsePauliOp, t_step, K): """ Return list of (coeff, pauli_label) for exp(H t_step) ~ sum_k (t_step^k/k!) H^k. """ terms = [] n_q = H_pauli.num_qubits Hk = SparsePauliOp.from_list([("I" * n_q, 1.0)]) # H^0 = I fact = 1.0 </pre>	<pre> if __name__ == "__main__": # 1. Forward Cole-Hopf transform on the initial condition psi0 = cole_hopf_forward(u_init(x)) psi_norm = np.linalg.norm(psi0) psi0_unit = psi0 / psi_norm T_FINAL = N_STEPS * dt # 2. Run Segmented LCU solver # K=6 (Taylor order), n_segments=30 (keeps H * dt_seg ~ 0.25) psi_final_unit, A, p_succ, n_terms = lcu_apply_segmented(H, T_FINAL, psi0_unit, K=6, n_segments=30) psi_final = psi_final_unit * psi_norm # 3. Inverse Cole-Hopf transform back to velocity space u(x, T) u_lcu = cole_hopf_inverse(psi_final.real, nu=NU) # 4. Compare against classical FTCS ground truth l2 = np.sqrt(np.mean((u_hist[-1] - u_lcu) ** 2)) print(f"LCU terms per segment : {n_terms}") print(f"Segmented LCU 1-norm (A) : {A:.4e}") print(f"LCU joint success prob : {p_succ:.4e}") print(f"L2 error vs classical : {l2:.2e}") </pre>
--	--

APPENDIX 5

pinn.py

<pre> import numpy as np import torch import torch.nn as nn from params import * # x, dx, dt, N, N_STEPS, NU, u_init torch.manual_seed(0) #device = torch.device("cuda" if torch.cuda.is_available() else "cpu") device = torch.device('cuda' if torch.cuda.is_available() else ('mps' if torch.backends.mps.is_available() else 'cpu')) print(f"Using device: {device}") X_MIN, X_MAX = float(x[0]), float(x[-1]) T_FINAL = N_STEPS * dt class PINN(nn.Module): def __init__(self, n_hidden=64, n_layers=4): super().__init__() layers = [nn.Linear(2, n_hidden), nn.Tanh()] for _ in range(n_layers - 1): layers += [nn.Linear(n_hidden, n_hidden), nn.Tanh()] layers += [nn.Linear(n_hidden, 1)] self.net = nn.Sequential(*layers) def forward(self, x, t): return self.net(torch.cat([x, t], dim=1)) def pde_residual(model, x_f, t_f, nu): x_f = x_f.clone().requires_grad_(True) </pre>	<pre> t_f = t_f.clone().requires_grad_(True) u = model(x_f, t_f) u_x = torch.autograd.grad(u, x_f, grad_outputs=torch.ones_like(u), create_graph=True)[0] u_t = torch.autograd.grad(u, t_f, grad_outputs=torch.ones_like(u), create_graph=True)[0] u_xx = torch.autograd.grad(u_x, x_f, grad_outputs=torch.ones_like(u_x), create_graph=True)[0] return u_t + u * u_x - nu * u_xx def train_pinn(n_epochs=5000, n_f=2000, lr=1e-3, verbose=True): model = PINN().to(device) opt = torch.optim.Adam(model.parameters(), lr=lr) # interior collocation points, random in space-time x_f = (torch.rand(n_f, 1) * (X_MAX - X_MIN) + X_MIN).to(device) t_f = (torch.rand(n_f, 1) * T_FINAL).to(device) # initial condition: t = 0, x = grid loss_pde = torch.mean(pde_residual(model, x_f, t_f, NU) ** 2) loss_ic = torch.mean((model(x_ic, t_ic) - u_ic) ** 2) loss_bc = torch.mean(model(x_bc_l, t_bc) ** 2) + torch.mean(model(x_bc_r, t_bc) ** 2) loss = loss_pde + 10.0 * loss_ic + 10.0 * loss_bc loss.backward() opt.step() </pre>
--	--

```

if verbose and epoch % 500 == 0:

    print(f"epoch {epoch:5d}  loss {loss.item():.3e}  "

          f"(pde {loss_pde.item():.2e}, ic
{loss_ic.item():.2e}, bc {loss_bc.item():.2e})")

    return model

def evaluate_pinn(model, x_grid, t_final):

    model.eval()

    x_t = torch.tensor(x_grid, dtype=torch.float32).view(-1,
1).to(device)

    t_t = torch.full_like(x_t, t_final)

    with torch.no_grad():

        u = model(x_t, t_t).cpu().numpy().flatten()

    return u

if __name__ == "__main__":

    from classical import u_hist

    model = train_pinn()

    u_pinn_final = evaluate_pinn(model, x, T_FINAL)

    l2 = np.sqrt(np.mean((u_hist[-1] - u_pinn_final) ** 2))

    print(f"L2 error (PINN vs classical): {l2:.2e}")

```

APPENDIX 6

compare.py

<pre> import numpy as np import matplotlib.pyplot as plt from scipy.linalg import expm from params import * from classical import u_hist from colehopf import cole_hopf_forward, cole_hopf_inverse, psi0 from hamiltonian import H, T_matrix from lcu import lcu_apply_segmented from pinn import train_pinn, evaluate_pinn T_FINAL = N_STEPS * dt u_ref = u_hist[-1] #classical def l2(u): return np.sqrt(np.mean((u_ref - u) ** 2)) psi_norm = np.linalg.norm(psi0) psi0_unit = psi0 / psi_norm #psi_exact_unit = expm(T_matrix * T_FINAL) @ psi0_unit #u_quantum_exact = cole_hopf_inverse(psi_exact_unit * psi_norm, nu=NU) psi_lcu_unit, A, p_succ, n_terms = lcu_apply_segmented(H, T_FINAL, psi0_unit, K=6) u_quantum_lcu = cole_hopf_inverse(psi_lcu_unit * psi_norm, nu=NU) # --- PINN --- pinn_model = train_pinn(n_epochs=3000) u_pinn = evaluate_pinn(pinn_model, x, T_FINAL) </pre>	<pre> # --- report --- print("\n=== L2 error comparison ===") #print(f'{'Cole-Hopf + exact exp(Tt)':30s}: {l2(u_quantum_exact):.3e}') print(f'{'Cole-Hopf + LCU (K=6)':30s}: {l2(u_quantum_lcu):.3e}') #f'[A={A:.2f}, success_prob={p_succ:.2e}, {n_terms} unitary terms]') print(f'{'PINN':30s}: {l2(u_pinn):.3e}') # --- plot --- plt.figure(figsize=(7, 3)) plt.plot(x, u_ref, 'k-', lw=2.5, label='Classical (Ref)') plt.plot(x, u_quantum_lcu, 'g-.', lw=2, label='Quantum (Cole-Hopf + LCU)') plt.plot(x, u_pinn, 'r:', lw=2.5, label='PINN') plt.xlabel('x') plt.ylabel(f'u(x, T={T_FINAL:.3f})') plt.title('Classical, Quantum, and Neural Network L2 Comparison') plt.legend() plt.tight_layout() plt.savefig('comparison.png', dpi=150) plt.show() </pre>
---	--

REFERENCES

- [1] J. D. Cole, "On a quasi-linear parabolic equation occurring in aerodynamics," *Quarterly of Applied Mathematics*, vol. 9, no. 3, pp. 225–236, 1951.
- [2] E. Hopf, "The partial differential equation $u_t + uu_x = \mu u_{xx}$," *Communications on Pure and Applied Mathematics*, vol. 3, no. 3, pp. 201–230, 1950.
- [3] R. Haberman, *Applied Partial Differential Equations*, 5th ed. Pearson Education, 2013.
- [4] M. T. Heath, *Scientific Computing: An Introductory Survey*, 2nd ed. McGraw-Hill, 2002.
- [5] M. A. Nielsen and I. L. Chuang, *Quantum Computation and Quantum Information*, 10th Anniversary Edition. Cambridge University Press, 2010.
- [6] S. Lloyd, "Universal Quantum Simulators," *Science*, vol. 273, no. 5278, pp. 1073–1078, 1996.
- [7] Qiskit Development Team, *Qiskit: An Open-Source Framework for Quantum Computing*. Available: IBM Quantum. [Qiskit Documentation](#)
- [8] H. Liao, "An implicit fourth-order compact finite difference scheme for one-dimensional Burgers' equation," *Applied Mathematics and Computation*, vol. 206, no. 2, pp. 755–764, 2008.